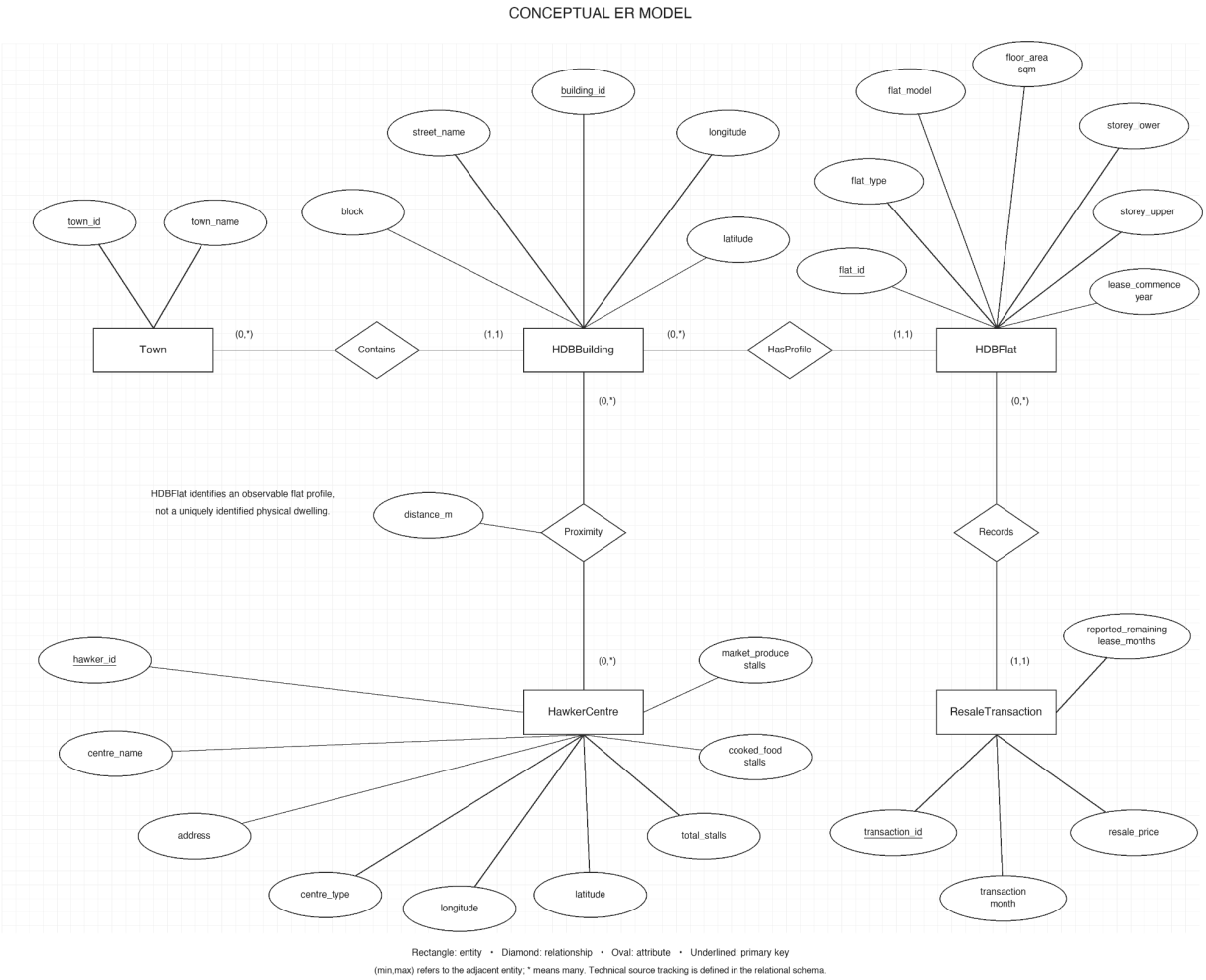
Task 1: Dataset-Verified Design

1. Conceptual ER model



2. Relational schema

Following W4, five entity tables and four relationship tables are first identified. The three 1:N relationships are merged into their many-side tables; the M:N Proximity relationship remains a separate table. The result is six base relations. Underlining denotes the primary key, arrows denote foreign keys, and ? denotes nullable.

Town(<u>town_id</u>, town_name)

HDBBuilding(<u>building_id</u>, town_id → Town, block, street_name, longitude?, latitude?)

HDBFlat(flat_id, building_id → HDBBuilding, flat_type, flat_model, floor_area_sqm, storey_lower, storey_upper, lease_commence_year)

ResaleTransaction(transaction_id, flat_id → HDBFlat, transaction_month, resale_price, reported_remaining_lease_months?, source_sha256, source_row_number)

HawkerCentre(hawker_id, centre_name, address?, centre_type?, longitude?, latitude?, total_stalls?, cooked_food_stalls?, market_produce_stalls?)

BuildingHawkerDistance(building_id → HDBBuilding, hawker_id → HawkerCentre, distance_m)

The two underlined distance-table attributes form one composite key. Alternate keys are town_name; (block,street_name); the complete flat-profile combination; and (source_sha256,source_row_number). The source hash and row number are loading metadata, not raw business fields.

Under the stated FDs, every nontrivial determinant is a superkey, so the base relations satisfy BCNF and therefore 3NF. Closures and minimal keys were checked. Splitting combined transaction/building/town data along their ID dependencies is lossless because each intersection is a key of an extracted relation (W5).

3. Proximity and missing values

Haversine straight-line distances use building and centre coordinates. The chosen classes are Near ≤500 m, Medium >500–1,000 m and Far >1,000 m. These thresholds are a project choice. All valid geocoded pairs are retained; classes are calculated in views.

Q4 assigns each building its nearest centre. Q5 selects its nearest centre with known market_produce_stalls >0. Equal-distance ties use the smaller hawker_id, preventing duplicate transaction counts. The supplied centre cohort includes market-only sites; centre_type supports an explicit alternative filter if the team chooses one.

Years-only leases in 2015–2016 are converted to months with year precision disclosed; 2017-onward year/month strings are parsed into total months. Pre-2015 leases remain unknown: Q2 needs an UNKNOWN bucket or a clearly disclosed reported-only cohort.

（For example:

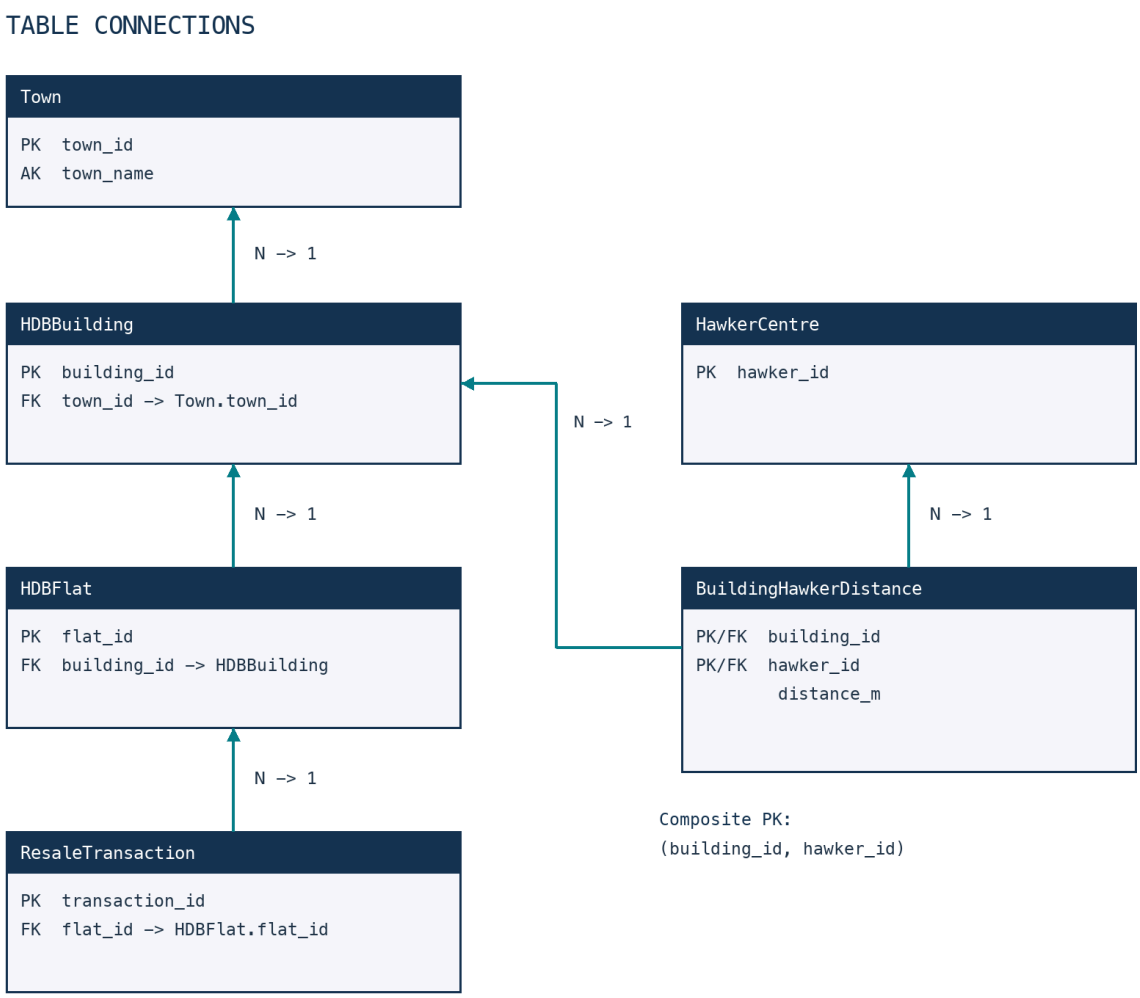
Block	**Street**	**Hawker ID**	**Distance KM**	**ProximityClass**
866	Jurong West St 81	1	0.6	Near
866	Jurong West St 81	2	1.2	Medium
866	Jurong West St 81	3	2.1	Far

Why not choose other distance calculation methods?

)

4. Table connections

Arrows point from FK to referenced PK. The distance bridge implements the building-to-centre M:N relationship; descriptive attributes are omitted from this overview.



5. SQL and validation

See in the SQL Scripts

SQL scripts

Run the sections in order. Create the database separately, reconnect to hdb_hawker_verified, then execute the schema and views. Blue monospaced text is executable SQL.

0. Create the database

Execute once against an administrative database, outside any transaction.

```
-- Execute once against an administrative database, outside a transaction.
CREATE DATABASE hdb_hawker_verified;
```

1. Initialize the schema

Connect to hdb_hawker_verified first. The following sections form one schema-creation transaction.

```
-- PostgreSQL schema; run connected to hdb_hawker_verified.
-- No extensions required. No experimental performance indexes are created.
-- Fail atomically if the dedicated schema already exists; never drop user
data.
BEGIN;
CREATE SCHEMA housing;
```

2. Town

Each town has a generated primary key and a unique canonical name.

```
CREATE TABLE housing.town (
    town_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    town_name text NOT NULL UNIQUE,
    CONSTRAINT town_name_canonical CHECK (
        town_name <> '' AND town_name = upper(btrim(town_name)))
);
```

3. HDB Building

Each building belongs to one town. Block and street name form an alternate key; coordinates may be unknown.

```
CREATE TABLE housing.hdb_building (
    building_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    town_id bigint NOT NULL REFERENCES housing.town(town_id) ON DELETE
RESTRICT,
    block text NOT NULL,
    street_name text NOT NULL,
    longitude double precision,
    latitude double precision,
```

```
    CONSTRAINT building_address_unique UNIQUE (block, street_name),
    CONSTRAINT block_canonical CHECK (block <> '' AND block =
upper(btrim(block))),
    CONSTRAINT street_canonical CHECK (
        street_name <> '' AND street_name = upper(btrim(street_name))),
    CONSTRAINT building_coordinate_pair CHECK (
        (longitude IS NULL AND latitude IS NULL) OR
        (longitude IS NOT NULL AND latitude IS NOT NULL)),
    CONSTRAINT building_longitude_range CHECK (longitude BETWEEN -180 AND 180),
    CONSTRAINT building_latitude_range CHECK (latitude BETWEEN -90 AND 90)
);
```

4. HDB Flat Profile

This table identifies observable attribute profiles. Lease commencement stays at profile level because a building can have multiple lease years.

```
CREATE TABLE housing.hdb_flat (
    flat_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    building_id bigint NOT NULL REFERENCES housing.hdb_building(building_id) ON
DELETE RESTRICT,
    flat_type text NOT NULL,
    flat_model text NOT NULL,
    floor_area_sqm numeric(10,3) NOT NULL,
    storey_lower smallint NOT NULL,
    storey_upper smallint NOT NULL,
    lease_commence_year smallint NOT NULL,
    CONSTRAINT flat_profile_unique UNIQUE (
        building_id, flat_type, flat_model, floor_area_sqm,
        storey_lower, storey_upper, lease_commence_year),
    CONSTRAINT flat_type_canonical CHECK (
        flat_type IN ('1 ROOM','2 ROOM','3 ROOM','4 ROOM','5 ROOM',
                      'EXECUTIVE','MULTI-GENERATION')),
    CONSTRAINT flat_model_canonical CHECK (
        flat_model <> '' AND flat_model = upper(btrim(flat_model))),
    CONSTRAINT flat_area_positive CHECK (
        floor_area_sqm > 0 AND floor_area_sqm < 'Infinity'::numeric),
    CONSTRAINT flat_storey_range CHECK (storey_lower >= 1 AND storey_upper >=
storey_lower),
    CONSTRAINT flat_lease_year CHECK (lease_commence_year BETWEEN 1 AND 9999)
);
```

5. Resale Transaction

Each transaction belongs to one profile. Missing reported lease remains NULL; file hash and row number preserve source traceability.

```
CREATE TABLE housing.resale_transaction (
    transaction_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    flat_id bigint NOT NULL REFERENCES housing.hdb_flat(flat_id) ON DELETE
RESTRICT,
    transaction_month date NOT NULL,
    resale_price numeric(14,2) NOT NULL,
```

```
    -- NULL means not reported; never silently replace missing values with
zero.
    reported_remaining_lease_months integer,
    -- SHA-256 of the exact source file and 1-based data row (header excluded).
    source_sha256 text NOT NULL,
    source_row_number bigint NOT NULL,
    CONSTRAINT transaction_source_unique UNIQUE (source_sha256,
source_row_number),
    CONSTRAINT transaction_month_start CHECK (
        isfinite(transaction_month) AND EXTRACT(DAY FROM transaction_month) =
1),
    CONSTRAINT transaction_price_positive CHECK (
        resale_price > 0 AND resale_price < 'Infinity'::numeric),
    CONSTRAINT transaction_lease_nonnegative CHECK
(reported_remaining_lease_months >= 0),
    CONSTRAINT transaction_source_hash CHECK (source_sha256 ~
'^[0-9a-f]{64}$'),
    CONSTRAINT transaction_source_row_positive CHECK (source_row_number >= 1)
);
```

6. Hawker Centre

The generated hawker_id identifies a centre. Source-backed type and stall counts are retained; unmatched values remain NULL.

```
CREATE TABLE housing.hawker_centre (
    hawker_id bigint GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    centre_name text NOT NULL,
    address text,
    centre_type text,
    longitude double precision,
    latitude double precision,
    -- Counts from the specified NEA snapshot; unmatched centres retain NULL.
    total_stalls integer,
    cooked_food_stalls integer,
    market_produce_stalls integer,
    CONSTRAINT hawker_centre_type CHECK (centre_type IN ('HC','MHC','MK')),
    CONSTRAINT hawker_name_nonempty CHECK (btrim(centre_name) <> ''),
    CONSTRAINT hawker_address_nonempty CHECK (address IS NULL OR btrim(address)
<> ''),
    CONSTRAINT hawker_coordinate_pair CHECK (
        (longitude IS NULL AND latitude IS NULL) OR
        (longitude IS NOT NULL AND latitude IS NOT NULL)),
    CONSTRAINT hawker_longitude_range CHECK (longitude BETWEEN -180 AND 180),
    CONSTRAINT hawker_latitude_range CHECK (latitude BETWEEN -90 AND 90),
    CONSTRAINT hawker_stalls_nonnegative CHECK (
        total_stalls >= 0 AND cooked_food_stalls >= 0 AND market_produce_stalls
>= 0),
    CONSTRAINT hawker_cooked_within_total CHECK (cooked_food_stalls <=
total_stalls),
    CONSTRAINT hawker_market_within_total CHECK (market_produce_stalls <=
total_stalls)
);
```

7. Building–Hawker Distance

The M:N Proximity relationship becomes a separate table. The composite primary key is (building_id, hawker_id); distance is stored in metres.

```
CREATE TABLE housing.building_hawker_distance (
    building_id bigint NOT NULL REFERENCES housing.hdb_building(building_id) ON
DELETE RESTRICT,
    hawker_id bigint NOT NULL REFERENCES housing.hawker_centre(hawker_id) ON
DELETE RESTRICT,
    distance_m double precision NOT NULL,
    PRIMARY KEY (building_id, hawker_id),
    CONSTRAINT distance_finite_nonnegative CHECK (
        distance_m >= 0 AND distance_m < 'Infinity'::double precision)
);
```

8. Metadata and commit

These comments document modelling and source decisions. Commit finishes the schema transaction.

```
COMMENT ON SCHEMA housing IS 'Task 1 revised against the eight local Datasets
CSV files; no fabricated unit identifiers or reported lease values.';
COMMENT ON TABLE housing.hdb_flat IS 'Observable profile; multiple physical
units may share one profile. No unit-level linkage is claimed.';
COMMENT ON TABLE housing.building_hawker_distance IS 'Haversine metres,
spherical radius 6371008.8 m; fixed source snapshots; ETL owns completeness and
refresh.';
COMMENT ON COLUMN housing.hawker_centre.market_produce_stalls IS 'NEA
no_of_mkt_produce_stalls; NULL=unknown, zero=known absent, >0=fresh-market
proxy.';
COMMENT ON COLUMN housing.hawker_centre.centre_type IS 'NEA type_of_centre;
NULL for an unmatched geo-only record.';
COMMENT ON COLUMN housing.resale_transaction.source_sha256 IS 'Exact
source-file SHA-256; freeze snapshots and keep filename/URL/date in an external
manifest.';
COMMIT;
```

9. Derived views and proximity classes

Run after the schema transaction. This creates one classification function and four views, including nearest-centre selection and transaction metrics.

```
-- Optional derived views; run AFTER 01_schema.sql. Not additional base
relations.
BEGIN;
-- Fixed, documented scheme; derive the class rather than storing a redundant
FD.
CREATE FUNCTION housing.proximity_class(distance_m double precision)
RETURNS text LANGUAGE sql IMMUTABLE STRICT PARALLEL SAFE AS $$
    SELECT CASE
        WHEN distance_m < 0 OR NOT (distance_m < 'Infinity'::double precision)
THEN NULL
```

```
        WHEN distance_m <= 500 THEN 'NEAR'
        WHEN distance_m <= 1000 THEN 'MEDIUM'
        ELSE 'FAR'
    END
$$;

CREATE VIEW housing.v_building_hawker_proximity AS
SELECT d.building_id, d.hawker_id, d.distance_m,
       housing.proximity_class(d.distance_m) AS proximity_class
FROM housing.building_hawker_distance d;

-- Deterministic tie-break; at most one row per building. Missing coverage is
absent,
-- so analytical queries must LEFT JOIN from buildings/transactions to report
it.
CREATE VIEW housing.v_nearest_hawker AS
SELECT DISTINCT ON (building_id) building_id, hawker_id, distance_m,
proximity_class
FROM housing.v_building_hawker_proximity
ORDER BY building_id, distance_m, hawker_id;

CREATE VIEW housing.v_nearest_market_hawker AS
SELECT DISTINCT ON (p.building_id)
       p.building_id, p.hawker_id, p.distance_m, p.proximity_class
FROM housing.v_building_hawker_proximity p
JOIN housing.hawker_centre h ON h.hawker_id = p.hawker_id
WHERE h.market_produce_stalls > 0
ORDER BY p.building_id, p.distance_m, p.hawker_id;

-- Missing reported lease remains NULL: no assumed 99-year term is inserted.
CREATE VIEW housing.v_transaction_metrics AS
SELECT t.*, f.building_id, f.flat_type, f.flat_model, f.floor_area_sqm,
       f.storey_lower, f.storey_upper, f.lease_commence_year,
       t.resale_price / f.floor_area_sqm AS price_per_sqm,
       t.reported_remaining_lease_months AS remaining_lease_months,
       CASE WHEN t.reported_remaining_lease_months IS NULL THEN 'NOT_REPORTED'
            WHEN t.transaction_month < DATE '2017-01-01' THEN 'REPORTED_YEARS'
            ELSE 'REPORTED_YEAR_MONTHS' END AS lease_value_method,
       CASE WHEN t.transaction_month < DATE '2012-03-01' THEN 'APPROVAL'
            ELSE 'REGISTRATION' END AS transaction_date_basis
FROM housing.resale_transaction t
JOIN housing.hdb_flat f ON f.flat_id = t.flat_id;

COMMENT ON VIEW housing.v_nearest_market_hawker IS 'Nearest centre with known
positive market produce count; unknown centres are excluded, not treated as
zero.';
COMMIT;
```